BACKEND NOTES · PERFORMANCE & REAL-TIME

Redis, beyond the cache

We start with a viral post that crashes a database. We fix it with caching, deal with the new problems caching creates, and then use the same Redis for rate limiting, scaling Socket.IO and live leaderboards. Every example uses `ioredis`.

ioredis 5 · Express · Socket.IO 4 · Redis 7+ · 16 sections · 32 quizzes

REDIS-CLI · ONE REDIS, MANY JOBS

```
127.0.0.1:6379> SET lock:post:42 req-7 NX PX 5000
OK  ← only one request rebuilds
127.0.0.1:6379> ZINCRBY lb:weekly 50 diya
"145"  ← leaderboard
127.0.0.1:6379> SISMEMBER online:users 9
(integer) 1  ← presence
127.0.0.1:6379>
```

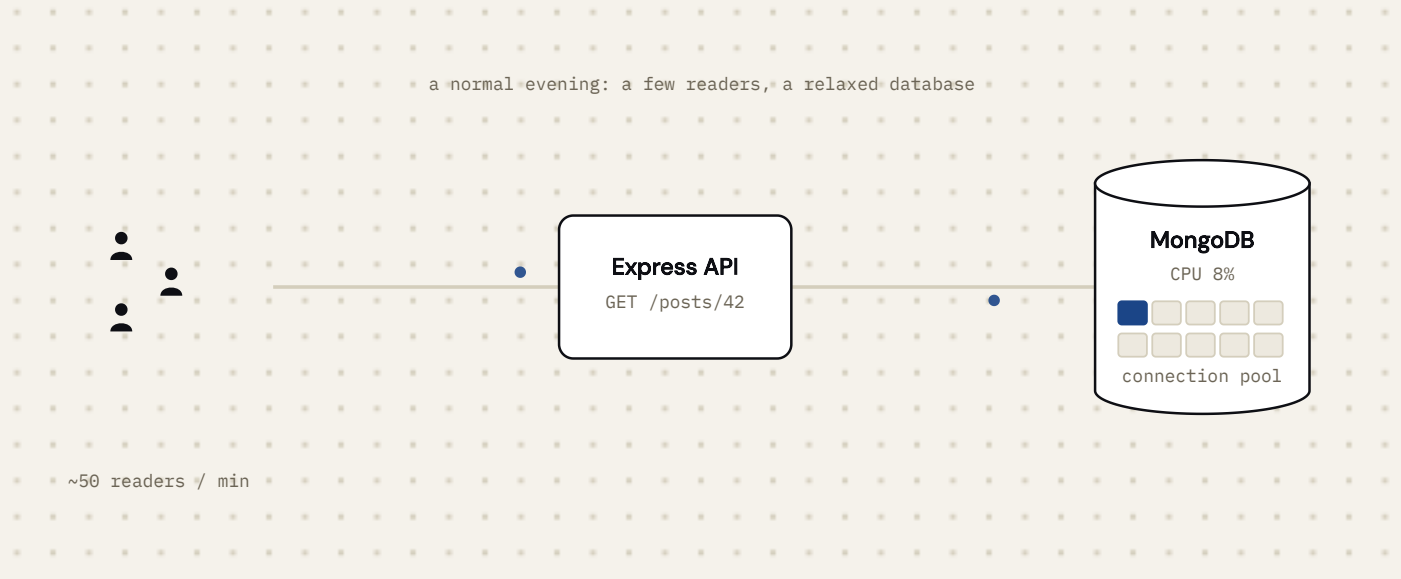
01 The viral post that killed the database

Start with the problem: lakhs of people reading the same data at the same time

Picture a social app built with Express and MongoDB, the kind you have already built in class. Users write posts. Others read, like and comment. Every time someone opens a post, the server runs `Post.findById(id)` and sends the result back.

One evening, a student from Bhopal posts a meme about exam results. It gets forwarded on WhatsApp groups and Instagram stories. Within ten minutes, **2 lakh people** open the same post. Step through what happens to the server.

STEP-THROUGH **From a quiet evening to a crashed database**



REQUESTS / SEC	DB CONNECTIONS	RESPONSE TIME	STATUS
≈ 1	3 / 100	20 ms	Healthy

A normal evening

About 50 people open post #42 per minute. Each view sends one request to Express, which runs one MongoDB query. The connection pool is almost empty and responses take ~20 ms.

← BACK STEP 1 / 6 ▶ AUTO-PLAY NEXT →

KEY CONCEPT

A **read-heavy hotspot** is a small piece of data that a huge number of users read at the same moment. The data barely changes, but the database pays the full price for every read. Product pages during a sale, results day, IPL live scores and trending posts are all read-heavy hotspots.

QUIZ · 01

In the viral post story, what actually crashed the database?

A — The post's data was too large for MongoDB to store

B — Lakhs of identical read requests used up every DB connection and all the CPU

QUIZ · 01

Why does the whole app go down, and not just the viral post page?

A — Login, feed and every other feature share the same database and its connection pool

B — Express automatically stops all routes when one route becomes slow

02 Why the database chokes

Work per read, connection pools, and the simple math behind the crash

Every read is real work

A database is built to store data safely on disk and to answer *any* question you can ask about it. That flexibility costs work on every query, even a simple `findById`. Watch both pipelines run. Each lit chip is one stage of work.

ANIMATION **What one read costs** illustrative pacing, not a benchmark

MongoDB · `Post.findById(42)` reads finished: **0**

borrow pool connection → parse query → plan · pick index → walk B-tree index → load page (RAM or disk) → serialise BSON → send

Redis · `GET post:42` reads finished: **0**

hash the key → look up in RAM dictionary → send bytes

The connection pool is a fixed number of counters

Your Node app doesn't open a fresh database connection per request. It keeps a **pool**. The MongoDB Node driver (which Mongoose uses) defaults to `maxPoolSize: 100`, and PostgreSQL defaults to `max_connections = 100`. A query holds one connection until it finishes. When all of them are busy, the next query waits in a queue.

> ANALOGY
>
> Think of a bank branch with a fixed number of counters and a token machine. If customers arrive faster than counters free up, the waiting hall fills. People who give up and come back (users hitting refresh) make the line even longer.

Do the math

The rough ceiling of any pool is **connections ÷ time per query**. With 100 connections and 20 ms per query, that's `100 ÷ 0.020 = 5,000` queries per second on a good day. A post page usually needs more than one query (post, author, top comments, "did I like this?"). Move the slider to see what viral traffic does to that ceiling.

INTERACTIVE **Load calculator**

100 connections · 20 ms/query · 4 queries per page view

READERS PER MINUTE



NO CACHE | REDIS CACHE · 95% HITS

READERS / MIN

2,00,000

requests / sec: **3,333**

DB queries needed / sec: **13,333**

reaching MongoDB: **13,333** / sec · capacity ≈ **5,000**

DATABASE UTILISATION

267% · over capacity

CRASHING

est. response: **keeps growing until timeouts**

💡 The numbers are illustrative. Real limits depend on hardware, indexes and query shape. What matters is the shape of the curve: **past 100% utilisation, waiting time doesn't grow slowly, it keeps growing until requests time out.** A 95% cache hit ratio drops the database back to a comfortable load.

So we want a copy of hot data that is (1) far cheaper to read than the database, (2) shared by every server instance we run, and (3) able to forget old data on its own. That's exactly the job Redis was built for.

QUIZ · 02

A pool has 100 connections and each query takes 50 ms. Roughly how many queries per second can it serve at best?

A — About 2,000

B — About 5,000

QUIZ · 02

When database utilisation goes past 100%, what happens to response time?

A — It rises a little and then levels off

B — Requests pile up in the queue and waiting time keeps growing until they time out

03 Enter Redis

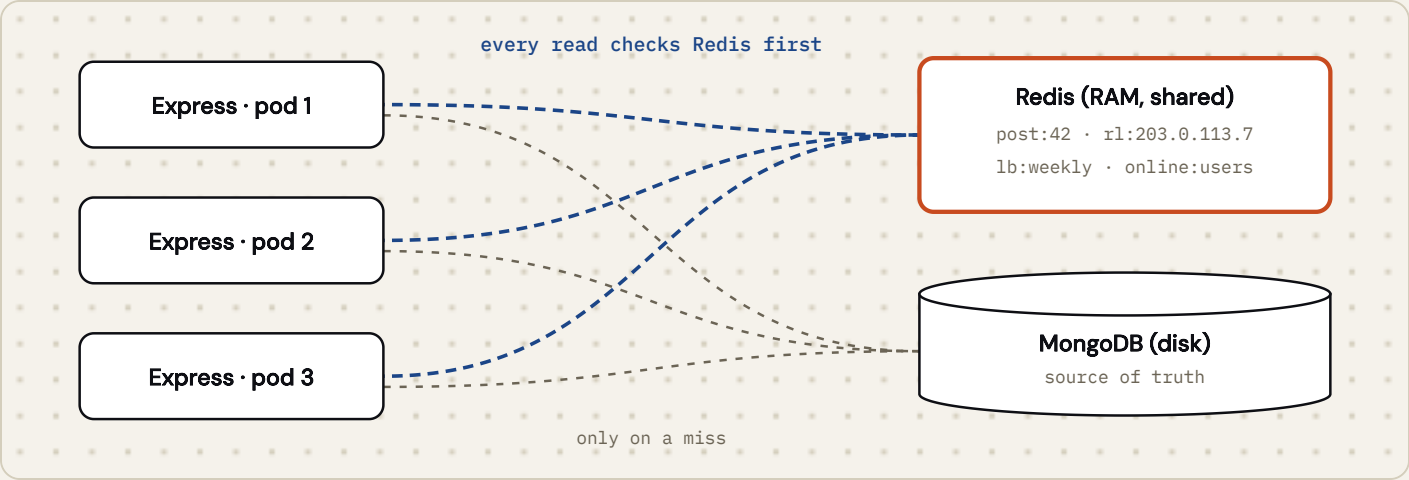
An in-memory data structure store that every server can share

KEY CONCEPT

Redis (REmote DIctionary Server) is a server that keeps data **in RAM** and lets you read and write it over the network by **key**. Picture a giant JavaScript `Map` running in its own process, which every one of your Node servers can talk to.

The chai tapri analogy. Brewing a fresh cup for every customer takes four minutes: boil water, add leaves, milk, sugar. That's a database query. A smart chaiwala keeps a big kettle of ready chai on the counter and pours in seconds. That's a **cache**. After half an hour the kettle chai goes cold and he makes a new batch (that's **TTL**, Section 6). If he changes the recipe but the kettle still has the old batch, customers get the old taste (that's **stale data**, Section 7).

Where Redis sits



Why Redis is so fast

- **Data lives in RAM.** No disk seek on the read path. A main-memory reference is around 100 ns, while an SSD random read is around 100 µs, roughly a thousand times slower.
- **Simple structures with known cost.** `GET` is a hash-table lookup, O(1). There's no query parser or planner deciding what to do.
- **One thread runs the commands.** Commands execute one at a time, so there are no locks, and every single command is **atomic**. That property powers rate limiters and leaderboards later.
- **Event-loop networking.** Like Node.js, one thread handles thousands of client connections with I/O multiplexing. A single small instance can usually serve tens of thousands to over a hundred thousand simple operations per second.

"Why not just use a JavaScript Map?"

Every student asks this. A `Map` inside your Node process works on your laptop and then falls apart in production.

What	In-process `Map`	Redis
Shared across servers	No. Each PM2 worker or Kubernetes pod has its own copy.	Yes. One store that every instance reads ✓
Survives an app restart / deploy	No. It's gone on every deploy.	Yes. Redis is a separate process ✓
Expiry	DIY with `setTimeout`	Built in: `EX`, `EXPIRE`, `TTL` ✓
Memory limits	Grows the Node heap and causes GC pauses	`maxmemory` + eviction policies ✓
Atomic counters across servers	Impossible	`INCR`, `ZINCRBY`, `MULTI` ✓
Data structures	Whatever you code yourself	Strings, hashes, lists, sets, sorted sets, streams, Pub/Sub ✓

⚠️ RAM is expensive and limited. Used as a cache, **Redis is not your source of truth**. Keep the real data in MongoDB or Postgres and treat Redis as a fast copy you can afford to lose. Redis can persist to disk (RDB snapshots, AOF logs), but caching code should still work if the cache comes back empty.

💡 In production you'll often use a managed service such as AWS ElastiCache, Redis Cloud or Upstash. **Valkey** is an open-source fork of Redis that speaks the same protocol. ioredis works with all of them.

QUIZ · 03

Redis runs commands one at a time on a single thread. What does that give us?

A — Commands never interleave, so each one is atomic without any locks

B — Node.js can only connect to single-threaded servers

QUIZ · 03

You run 4 Express instances with PM2 and cache posts in a JS `Map`. Post 42 is cached in instance 1, and the next request lands on instance 3. What happens?

A — Cache miss. Instance 3 has its own empty Map and queries the DB again

B — Instance 3 reads instance 1's Map automatically

04 Setting up Redis with ioredis

Run Redis in Docker, create one shared client, and learn the first commands

STEP 1 — RUN REDIS LOCALLY

TERMINAL COPY

```
docker run -d --name redis -p 6379:6379 redis:7-alpine
docker exec -it redis redis-cli ping
# PONG
```

FLAG	VALUE USED	WHAT IT DOES
`-d`	—	Runs the container in the background (detached).
`--name`	`redis`	Gives the container a name so `docker exec redis …` works.
`-p`	`6379:6379`	Maps host port → container port. 6379 is Redis's default port.
`image`	`redis:7-alpine`	Official Redis 7 image on a small Alpine base.

STEP 2 — INSTALL IOREDIS

TERMINAL COPY

```
npm install ioredis
```

STEP 3 — CREATE ONE SHARED CLIENT

Create the connection **once** in its own module and import it everywhere, the same way you connect Mongoose once in `db.js`.

LIB/REDIS.JS COPY

```
import { Redis } from "ioredis";

export const redis = new Redis(process.env.REDIS_URL ?? "redis://127.0.0.1:6379", {
  maxRetriesPerRequest: 2,      // fail a command after 2 retries instead of waiting fo
  // enableOfflineQueue: false, // fail instantly while disconnected (see Section 10)
});

redis.on("ready", () => console.log("✅ Redis ready"));
redis.on("error", (err) => console.error("❌ Redis error:", err.message));
redis.on("reconnecting", () => console.log("↻ Redis reconnecting…"));
```

OPTION	VALUE USED	WHAT IT DOES
REDIS_URL	redis://127.0.0.1:6379	Connection string. Managed services give you `rediss://user:pass@host:port` (double "s" = TLS).
maxRetriesPerRequest	2	How many reconnect attempts a pending command waits through before it rejects. The default is 20, which can leave HTTP requests hanging.
enableOfflineQueue	true (default)	While disconnected, ioredis queues commands and sends them after reconnecting. Set it to `false` to fail fast instead.
"error" listener	required	Without it, connection errors become unhandled error events and get noisy in your logs.

STEP 4 — FIRST COMMANDS

ioredis method names are the Redis command names in lowercase, with the arguments in the same order you'd type in `redis-cli`. Learn a command once and you can use it from both.

PLAYGROUND.JS COPY

```
import { redis } from "./lib/redis.js";

await redis.set("greeting", "namaste");                // "OK"
await redis.get("greeting");                           // "namaste"
await redis.get("missing");                            // null   ← not undefined
await redis.set("otp:9876543210", "4821", "EX", 300); // expires in 5 minutes
await redis.incr("post:42:views");                     // 1, then 2, 3 …  (a number)
await redis.del("greeting");                           // 1  (how many keys were removed)

// Redis stores strings. Objects must be serialised.
await redis.set("post:42", JSON.stringify({ caption: "Hello Bhopal" }));
const post = JSON.parse(await redis.get("post:42"));
```

Naming keys

Redis has no tables. The convention is colon-separated names that read from general to specific, so keys group nicely and you can find them by pattern.

KEY	HOLDS	USED IN
post:42	Cached JSON of post 42	Caching (§5)
post:42:views	View counter	Counters (§11)
lock:post:42	"Someone is rebuilding post 42"	Stampede protection (§8)
rl:login:203.0.113.7	Login attempts from one IP	Rate limiting (§12)
online:users	Set of online user IDs	Presence (§14)
lb:weekly:2026-W43	This week's scores	Leaderboards (§15)

VERIFY — WATCH COMMANDS LIVE

Open a second terminal and run `MONITOR`. Every command your app sends shows up as it happens, which is the best way to see your caching logic working.

TERMINAL COPY

```
docker exec -it redis redis-cli monitor
# 1729500000.123 [0 172.17.0.1:52144] "GET" "post:42"
# 1729500000.161 [0 172.17.0.1:52144] "SET" "post:42" "{...}" "EX" "60"
```

⚠️ **Never write `new Redis()` inside a route handler.** It opens a new TCP connection per request and you'll run out of connections under load. It's the same mistake as connecting to MongoDB on every request. Also keep `MONITOR` for local debugging only. On a busy production server it slows Redis down.

QUIZ · 04

`await redis.get("post:999")` for a key that doesn't exist returns…

A — undefined

B — null

QUIZ · 04

Where should you create the ioredis client?

A — Once, in a shared module that every file imports

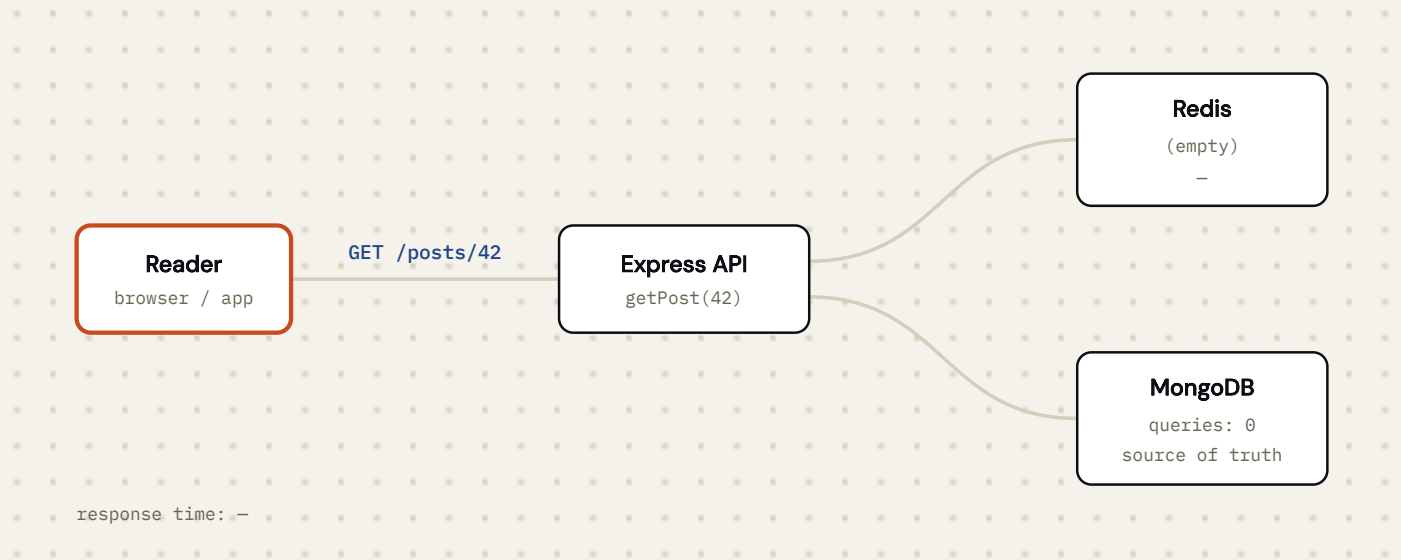
B — Inside each route handler, so every request gets a fresh connection

05 Caching with the cache-aside pattern

Check Redis first, fall back to the database, keep a copy for the next reader

The most common caching pattern is **cache-aside**, also called lazy loading. Your application sits in the middle and talks to both Redis and the database. Redis never talks to the database itself. Data only goes into the cache after somebody actually asks for it.

STEP-THROUGH **One miss, then hits for everyone else**



A request arrives

A reader opens `/posts/42`. Express calls our `getPost(42)` function.

← BACK

STEP 1 / 8

▶ AUTO-PLAY

NEXT →

The code

SERVICES/POST.SERVICE.JS COPY

```
import { redis } from "../lib/redis.js";
import Post from "../models/post.model.js";

const POST_TTL = 60; // seconds

export async function getPost(postId) {
  const key = `post:${postId}`;
```

```
  // 1. Try the cache first
  const cached = await redis.get(key);
  if (cached) return { post: JSON.parse(cached), source: "HIT" };

  // 2. Miss → ask the database
  const post = await Post.findById(postId).lean();
  if (!post) return { post: null, source: "MISS" };

  // 3. Keep a copy for the next reader, with an expiry
  await redis.set(key, JSON.stringify(post), "EX", POST_TTL);
  return { post, source: "MISS" };
}
```

ROUTES/POST.ROUTES.JS COPY

```
router.get("/posts/:id", async (req, res) => {
  const { post, source } = await getPost(req.params.id);
  res.set("X-Cache", source);              // handy while debugging: HIT or MISS
  if (!post) return res.status(404).json({ message: "Post not found" });
  res.json(post);
});
```

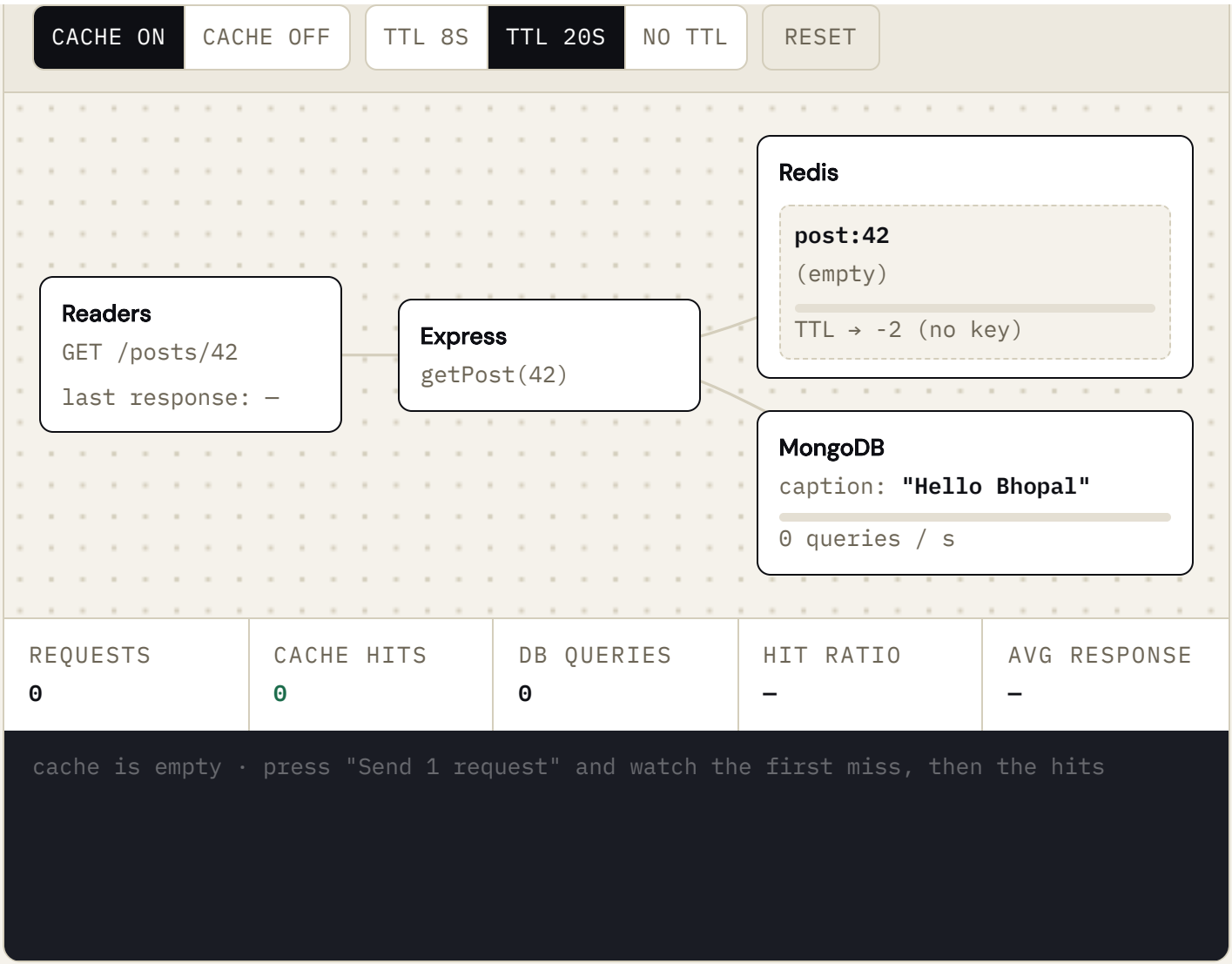
ARGUMENT	VALUE USED	WHAT IT DOES
key	`` `post:${postId}` ``	One key per post. The same ID always maps to the same key.
value	`JSON.stringify(post)`	Redis stores strings, so serialise objects. `.lean()` gives a plain object instead of a Mongoose document.
"EX"	`60`	Expire after 60 seconds. Use `"PX"` for milliseconds.

Try it: watch the hit ratio climb

Send requests one at a time, then fire a viral burst. Switch the cache off to feel what the database went through in Section 1.

SIMULATOR **Cache-aside, live** latencies are simulated

SEND 1 REQUEST VIRAL BURST ×30



💡 Fire a burst while the cache is **empty** and count the misses. Several requests miss before the first one finishes writing to Redis. That's a small **cache stampede**, and Section 8 is about stopping it.

KEY CONCEPT

Hit ratio = hits ÷ (hits + misses). At 95%, only 5 out of every 100 reads reach the database. For a hot key like our viral post, the hit ratio gets close to 100% after the first request.

What's worth caching?

Good candidates	Think twice
Read often, changes rarely: post content, product details, public profiles	Changes on almost every request (live stock count at checkout)

Good candidates	Think twice
Expensive to compute: trending list, aggregation results, recommendations	Read once, never again (one-off reports)
Same answer for many users: config, feature flags, category trees	Money or permissions where a stale answer causes harm (wallet balance, role checks)

OTHER CACHING PATTERNS YOU'LL HEAR ABOUT

Pattern	Who loads from the DB?	When is the cache updated?
Cache-aside (this section)	Your app, on a miss	Lazily on read, plus delete on write ✓ most common
Read-through	A cache library or layer, on a miss	Same idea, hidden behind a library
Write-through	—	On every write, the app writes the DB and the cache together
Write-behind	—	Writes go to Redis first and get flushed to the DB later (e.g. like counters)

QUIZ · 05

In cache-aside, who loads data from the database on a cache miss?

A — Redis fetches it from MongoDB automatically

B — Your application code, which then writes the result to Redis

QUIZ · 05

Out of 1,000 reads, 950 were served from Redis. What's the hit ratio, and how many reads reached the DB?

A — 95% hit ratio, 50 DB reads

B — 5% hit ratio, 950 DB reads

06 TTL: giving data an expiry date

How Redis forgets things on its own, and how to choose how long to remember

TTL (time to live) is a countdown attached to a key. When it reaches zero, Redis deletes the key. TTL does two jobs at once: it **caps how stale** a cached copy can get, and it **frees memory** automatically so the cache doesn't grow forever. Back at the chai tapri, it's the rule "make a fresh kettle every 30 minutes".

INTERACTIVE **TTL lab** a tiny Redis running in your browser

SET OTP … EX 10 | SET POST:42 … EX 30 | SET SITE:BANNER (NO TTL) | FLUSHALL

post:42

"{"caption":"Hello Bhopal"}"

45

TTL | GET | EXPIRE 30 | PERSIST

site:banner

"Diwali Sale"

∞

TTL | GET | EXPIRE 30 | PERSIST

```
127.0.0.1:6379> TTL post:42
(integer) 45
ioredis → await redis.ttl("post:42") // 45
```

The commands

REDIS-CLI COPY

```
127.0.0.1:6379> SET post:42 "{...}" EX 60       # set value + expiry together (seconds)
OK
127.0.0.1:6379> TTL post:42                      # seconds left
(integer) 57
127.0.0.1:6379> EXPIRE post:42 300               # change the expiry of an existing key
(integer) 1
127.0.0.1:6379> PERSIST post:42                  # remove the expiry: lives forever
(integer) 1
127.0.0.1:6379> TTL post:42
(integer) -1
127.0.0.1:6379> TTL post:999
(integer) -2
```

THE SAME IN IOREDIS COPY

```
await redis.set("post:42", json, "EX", 60);    // seconds
await redis.set("lock:post:42", id, "PX", 5000); // milliseconds
await redis.expire("post:42", 300);            // 1 if the key exists, 0 if not
await redis.ttl("post:42");                    // 57  |  -1 no expiry  |  -2 no such key
await redis.pttl("post:42");                   // same thing in milliseconds
await redis.persist("post:42");                // remove the expiry
await redis.set("post:42", newJson, "KEEPTTL"); // overwrite value, keep the old countdo
```

COMMAND / OPTION	EXAMPLE	WHAT IT DOES
SET … EX	EX 60	Value and expiry in one atomic command. Prefer this over a separate `SET` + `EXPIRE`.
SET … PX	PX 5000	Expiry in milliseconds, useful for short locks.
EXPIRE / PEXPIRE	EXPIRE k 300	Set or change the TTL on an existing key. Redis 7 adds `NX`, `XX`, `GT` and `LT` flags.
TTL / PTTL	TTL k	Remaining time. `-1` = key exists with no expiry, `-2` = key doesn't exist.

COMMAND / OPTION	EXAMPLE	WHAT IT DOES
PERSIST	`PERSIST k`	Removes the expiry.
KEEPTTL	`SET k v KEEPTTL`	Overwrite the value without resetting the countdown.

⚠️ **A plain `SET` wipes the TTL.** If `post:42` had `EX 60` and you run `SET post:42 newValue` without `EX`, the key now lives *forever* (`TTL` → `-1`). Always pass `EX` again, or use `KEEPTTL`. `INCR` is different: it keeps the existing TTL.

💡 **How Redis actually expires keys:** lazily (it checks the expiry whenever you access a key) and actively (a background job samples keys that have TTLs about 10 times a second and deletes expired ones). An expired key is never returned to you, but it may take a moment to leave memory.

Choosing a TTL

There's no magic number. A short TTL keeps data fresher but sends more traffic to the DB. A long TTL protects the DB but shows older data. Decide per kind of data:

DATA	TYPICAL TTL	WHY
Viral post body	`60 – 300 s`	Rarely edited. Even 60 s turns lakhs of reads into one query a minute.
Like / view counts	`5 – 10 s`	Nobody notices if a count is a few seconds behind.
Public user profile	`10 – 30 min`	Changes rarely. Delete it on edit (Section 7).
Feature flags / config	`30 s – 5 min`	Read on every request. Short enough that a change rolls out quickly.
OTP	`5 min`	Here the TTL is a security rule, not a performance setting.
Rate-limit window	`= window`	The TTL *is* the window (Section 12).

QUIZ · 06

`TTL post:42` returns `-1`. What does that mean?

A — The key exists but has no expiry. It lives until deleted or evicted

B — The key has already expired

QUIZ · 06

A key was set with `EX 60`. Ten seconds later you run `SET post:42 newValue` with no options. What's its TTL now?

A — Still about 50 seconds

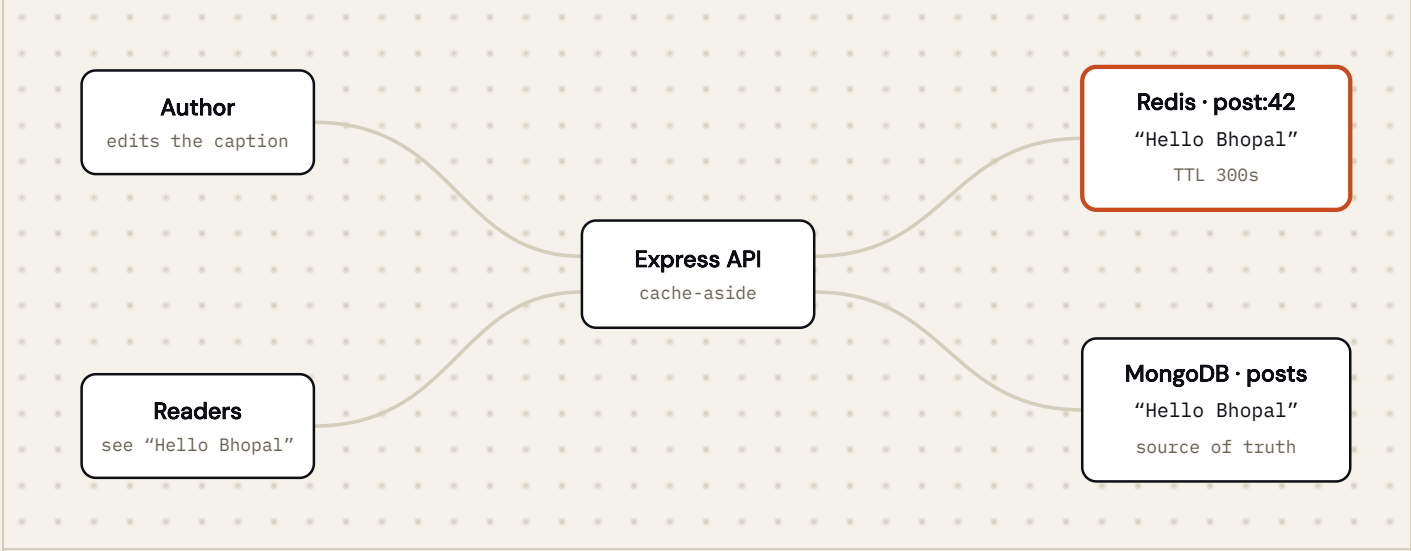
B — -1, because the plain SET removed the expiry

07 The first new problem: stale data

Every cache is a copy, and copies go out of date

Every solution in tech brings its own problems. Caching fixed the crash, but now the same data lives in **two places**, MongoDB and Redis, and nothing forces them to agree. When the author edits the post, MongoDB has the new caption while Redis keeps serving the old one. Readers get **stale data**.

STEP-THROUGH **How a cache goes stale, and the fix**



Everything is in sync

The post is cached with a 300-second TTL. Redis and MongoDB both say "Hello Bhopal". Readers get fast cache hits.

← BACK — STEP 1 / 6 — ▶ AUTO-PLAY — NEXT →

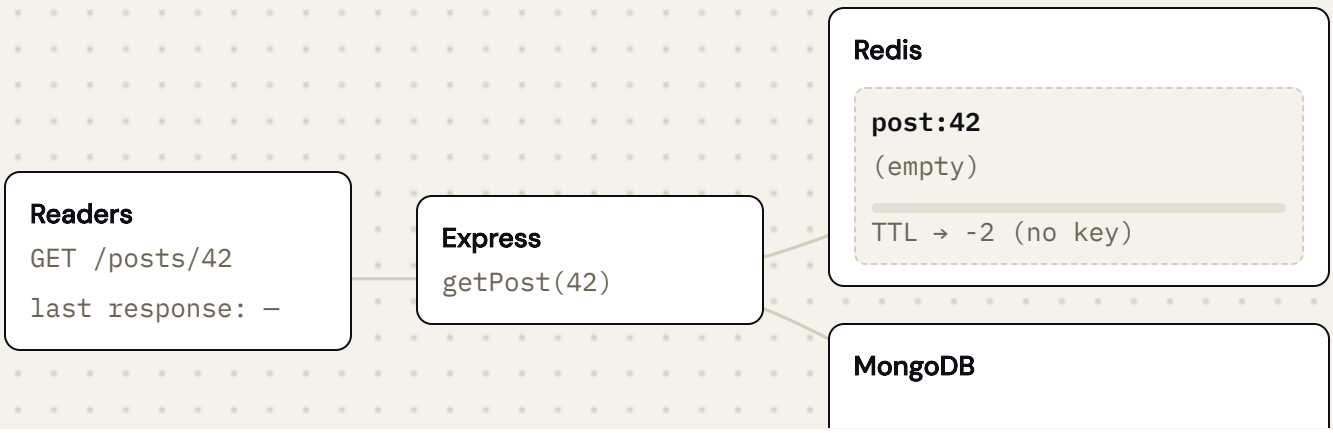
Try it: edit the post and watch readers

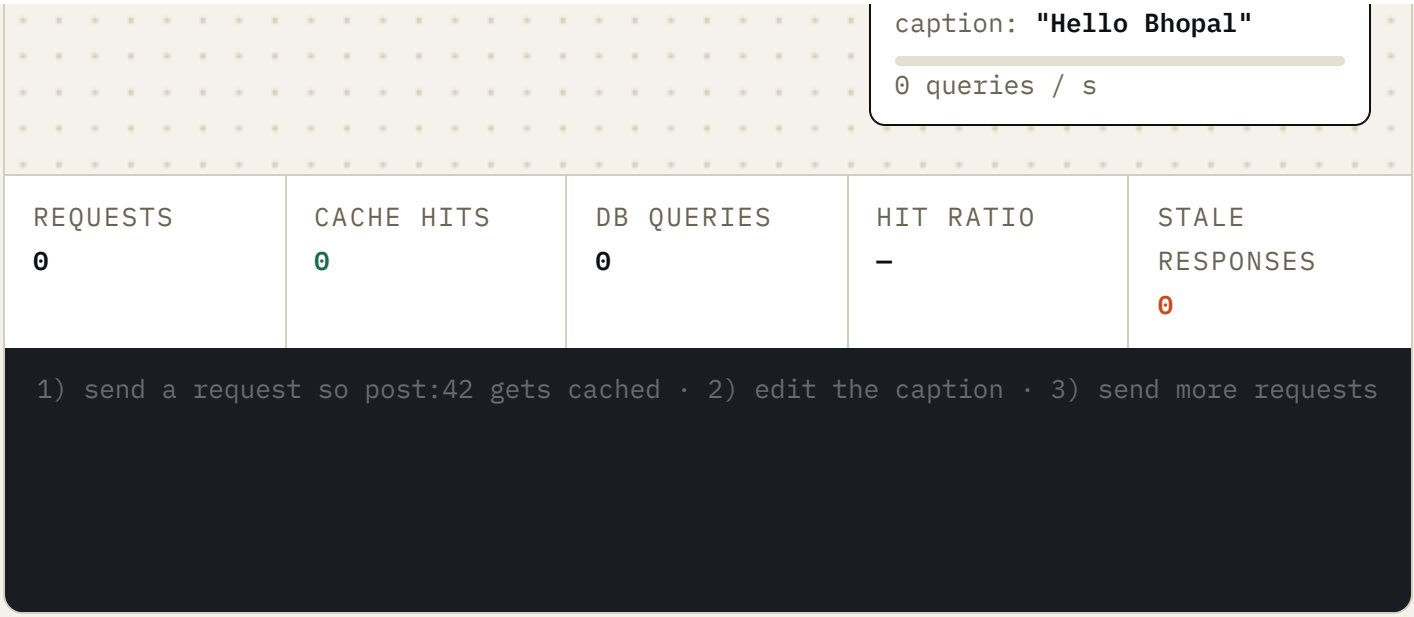
Send a few requests so the post gets cached, then edit the caption in the DB. With **TTL only**, responses stay stale until the key expires. Switch to **DEL on write** and edit again.

SIMULATOR Edit the post, then read it — latencies are simulated

SEND 1 REQUEST | READERS ×8 | EDIT CAPTION IN DB

TTL ONLY | DEL ON WRITE | TTL 8S | TTL 20S | NO TTL | RESET





Invalidation strategies

Strategy	How	Freshness	The catch
TTL only	Do nothing on writes. Let keys expire.	Stale for up to the TTL	Simplest. Fine for counts, feeds and trending lists.
Delete on write	Update DB → `DEL key`	Fresh on the next read ✓	You must know every key that holds that data.
Write-through	Update DB → `SET key newValue`	Fresh immediately	Two concurrent writers can SET out of order and leave the older value cached.
Versioned keys	Key includes a version: `post:42:v7`	Fresh as soon as the version bumps	Needs somewhere to store the version. Old versions take memory until they expire.

Order matters: update the DB, *then* delete the cache

It looks harmless to delete the cache first. Here's the race that bites you when a reader slips in between:

- **Writer:** `DEL post:42`
  Cache is empty. The DB update hasn't happened yet.
- **Reader:** `GET post:42` → **miss**

Goes to the database…

Reader: reads the old document from MongoDB

The writer's update is still in flight.

Writer: `updateOne(...)` finishes

The DB now has the new caption.

Reader: `SET post:42 <old> EX 300`

The old value is cached for 5 minutes, even though we "invalidated".

Updating the DB first and deleting afterwards shrinks this window to almost nothing. Keep a TTL anyway, as a safety net for the rare race that still slips through.

SERVICES/POST.SERVICE.JS COPY

```
export async function updatePost(postId, changes) {
  // 1. Source of truth first
  const post = await Post.findByIdAndUpdate(postId, changes, { new: true }).lean();
  if (!post) return null;

  // 2. Then remove every cached copy that contains this post
  await redis.del(`post:${postId}`, `user:${post.author}:posts`);
  return post;
}

export async function deletePost(postId) {
  await Post.findByIdAndDelete(postId);
  await redis.del(`post:${postId}`);
}
```

💡 Phil Karlton's famous line: *"There are only two hard things in Computer Science: cache invalidation and naming things."* In practice, ask a product question for each kind of data: **how stale can this be before someone notices or gets hurt?** A like count 10 s behind is fine. A caption 5 minutes old after an edit is annoying. A wallet balance stale at checkout is unacceptable, so don't cache it at all.

QUIZ · 07

An author edits a post. With cache-aside, what's the safest order?

A — Update the DB first, then DEL the cache key

B — DEL the cache key first, then update the DB

QUIZ · 07

Why do teams usually `DEL` the key on write instead of `SET`-ing the new value?

A — DEL is faster than SET

B — Concurrent writers can SET out of order and leave an old value. DEL just makes the next read fetch fresh data

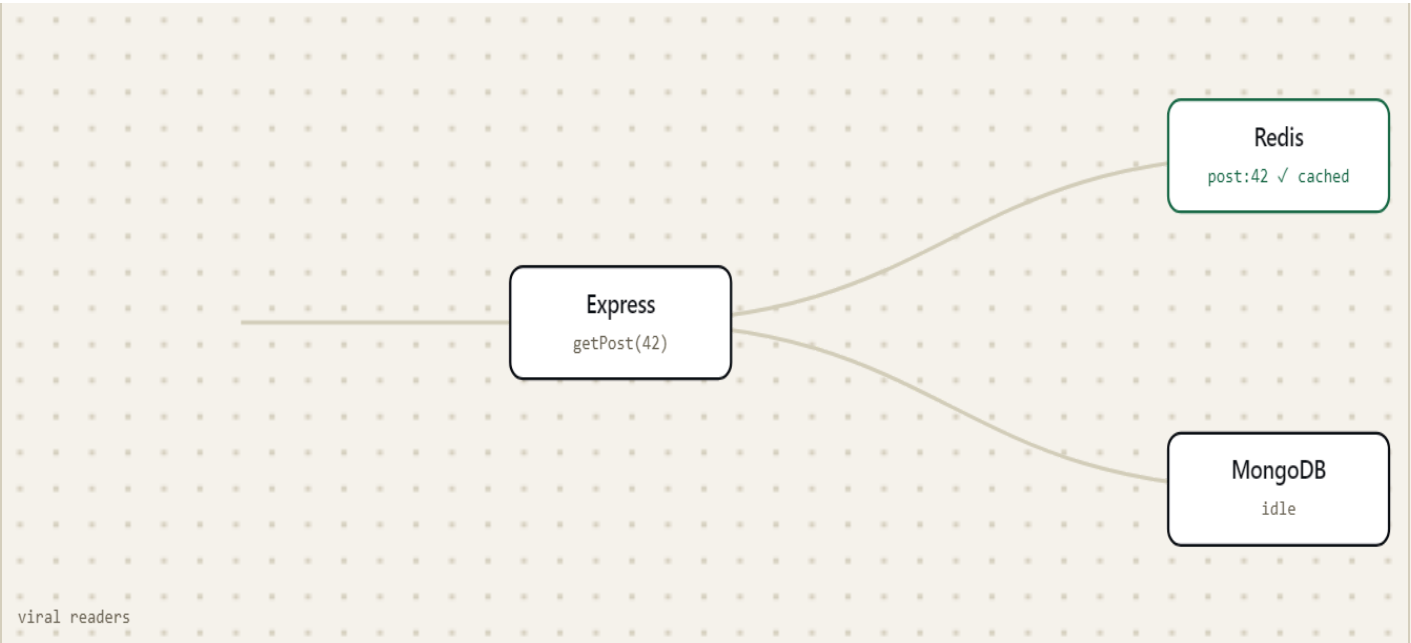
08 Cache stampede: when the hot key expires

Thousands of requests miss at the same moment and run the same query together

Back to our viral post, cached with a 60-second TTL. At second 60 the key disappears, and thousands of requests are in flight at that instant. They all check Redis, all get `nil`, and all run the same DB query. It's the exact crash from Section 1, triggered by our own TTL. This is called a **cache stampede** (also thundering herd or dog-piling).

SIMULATOR **A hot key expires under viral traffic** each dot is a request

EXPIRE THE HOT KEY | RESET COUNTERS | NO PROTECTION | MUTEX LOCK · SET NX



DB QUERIES THIS EXPIRY	IN FLIGHT AT DB	WAITING FOR LOCK	SERVED FROM CACHE
0	0	0	0

request · cache hit · DB query (miss) · lock owner rebuilding · waiting for the lock

Click **Expire the hot key** with no protection and watch the DB counter jump. Then switch to the mutex lock and expire it again: one request rebuilds while the others wait a few milliseconds and then read the fresh copy.

Fix 1 — Let only one request rebuild (mutex lock)

Redis can act as a lock because `SET … NX` ("only set if the key does Not eXist") is atomic. Out of thousands of simultaneous requests, exactly one gets `"OK"`. Everyone else gets `null`.

SERVICES/POST.SERVICE.JS · STAMPEDE-SAFE COPY

```
import { randomUUID } from "node:crypto";
import { redis } from "../lib/redis.js";
import Post from "../models/post.model.js";

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

// Delete the lock only if WE still own it (compare-and-delete, atomic in Lua)
const RELEASE_LOCK = `
  if redis.call("GET", KEYS[1]) == ARGV[1] then
```

```
    return redis.call("DEL", KEYS[1])
  end
  return 0
`;

export async function getPostSafe(postId, attempt = 0) {
  const key = `post:${postId}`;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const lockKey = `lock:${key}`;
  const token = randomUUID();
  const gotLock = await redis.set(lockKey, token, "PX", 5000, "NX"); // "OK" or null

  if (gotLock === "OK") {
    try {
      const post = await Post.findById(postId).lean();      // only ONE request does thi
      if (post) await redis.set(key, JSON.stringify(post), "EX", 60);
      return post;
    } finally {
      await redis.eval(RELEASE_LOCK, 1, lockKey, token);
    }
  }

  // Someone else is rebuilding: wait a little, then read the cache again
  if (attempt >= 20) return Post.findById(postId).lean();   // give up after ~1 s
  await sleep(50);
  return getPostSafe(postId, attempt + 1);
}
```

PART	VALUE USED	WHY IT'S THERE
lockKey	lock:post:42	One lock per cache key, so different posts rebuild in parallel.
token	randomUUID()	Marks who owns the lock, so we never delete a lock someone else took after ours expired.
"PX", 5000	5 s	If the owner crashes, the lock frees itself instead of blocking everyone forever.
"NX"	—	Set only if absent. This is what makes it a lock.
redis.eval	Lua script	GET + compare + DEL run as one atomic step inside Redis.

Fix 2 — Merge duplicate misses inside one server (single-flight)

Within one Node process you don't even need Redis for this. Keep the in-progress Promise and let concurrent callers wait on it. That turns 1,000 identical misses on one server into one DB query. Combine it with the Redis lock to also cover requests on *other* servers.

LIB/SINGLEFLIGHT.JS COPY

```
const inflight = new Map(); // key -> Promise

export function singleFlight(key, loader) {
  if (inflight.has(key)) return inflight.get(key);          // join the request already i
  const p = loader().finally(() => inflight.delete(key));
  inflight.set(key, p);
  return p;
}

// usage
const post = await singleFlight(`post:${id}`, () => getPostSafe(id));
```

Fix 3 — Refresh before it expires (stale-while-revalidate)

For the hottest keys, never let readers see a miss. Store a "fresh until" time inside the value and give the Redis key a longer real TTL. When a reader finds data that's past its fresh time, it still returns that slightly old copy right away and starts one background refresh, protected by the same lock.

SERVICES/POST.SERVICE.JS · SOFT EXPIRY COPY

```
const FRESH_MS = 60_000;    // logical freshness
const HARD_TTL = 300;       // real Redis TTL (seconds): the safety net

export async function getPostSWR(postId) {
  const key = `post:${postId}`;
  const raw = await redis.get(key);
  if (raw) {
    const { data, freshUntil } = JSON.parse(raw);
    if (Date.now() > freshUntil) refreshInBackground(postId);  // don't await
    return data;                                               // instant, maybe a bit
  }
  return refresh(postId);                                      // truly cold: load now
```

```
}

async function refresh(postId) {
  const data = await Post.findById(postId).lean();
  const value = JSON.stringify({ data, freshUntil: Date.now() + FRESH_MS });
  await redis.set(`post:${postId}`, value, "EX", HARD_TTL);
  return data;
}

async function refreshInBackground(postId) {
  const ok = await redis.set(`lock:post:${postId}`, "1", "PX", 5000, "NX");
  if (ok === "OK") refresh(postId).catch(console.error);      // only one refresher
}
```

Fix	Protects against	Cost
Mutex lock (SET NX PX)	Many servers rebuilding the same key	Waiters add a few ms of latency
Single-flight	Duplicate misses inside one process	Almost free ✓
Stale-while-revalidate	Readers ever seeing a miss on hot keys	Serves data up to one refresh cycle old

QUIZ · 08

`SET lock:post:42 abc NX PX 5000` returns `null`. What does that mean for this request?

A — Someone else holds the lock, so wait briefly and re-read the cache

B — Redis is down, so query the DB immediately

QUIZ · 08

Why does the lock itself have an expiry (`PX 5000`)?

A — If the lock owner crashes, the lock frees itself instead of blocking everyone forever

B — Redis requires every key to have a TTL

09 Cache avalanche & cache penetration

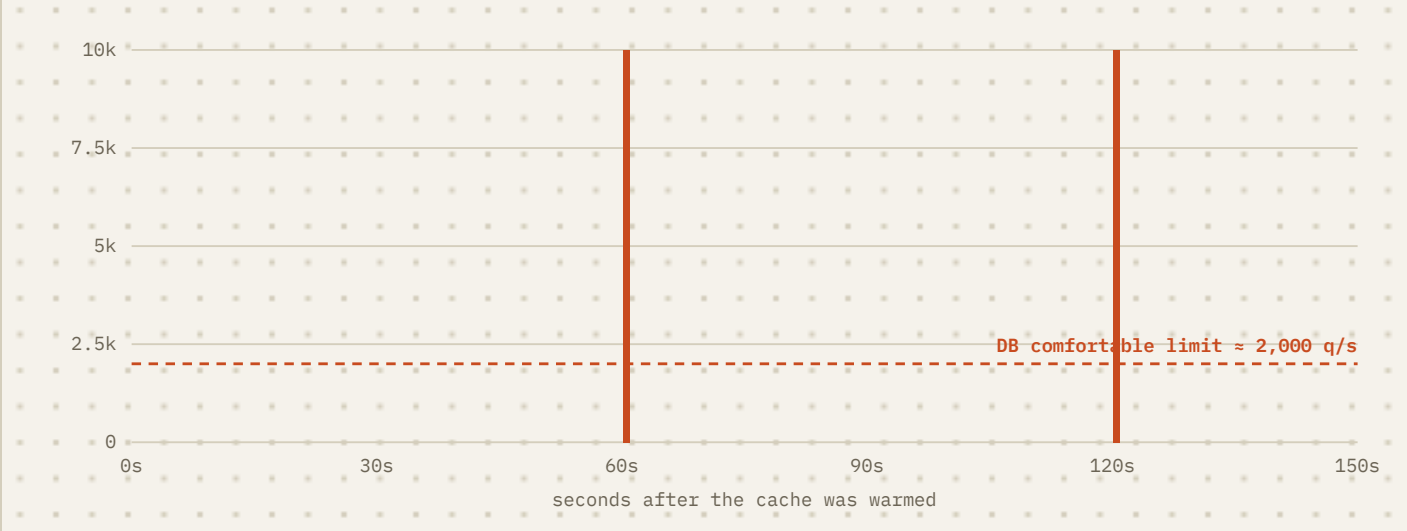
Many keys expiring together, and requests for data that doesn't exist

Avalanche: thousands of keys expire in the same second

A stampede is about *one* hot key. An **avalanche** is many keys expiring at once. Say a deploy (or a midnight cron job) warms the cache for 10,000 products, all with `EX 60`. Exactly 60 seconds later, all 10,000 vanish together and the database gets hit with 10,000 queries in a single second. Then again at 120 s. A Redis restart that empties the whole cache (a **cold cache**) does the same thing.

CHART **DB queries per second after warming 10,000 keys at t = 0**

SAME TTL · EX 60 | TTL + JITTER · EX 60–89 | REPLAY



Peak: **10,000 queries in one second** at 60s (and again at 120s). That's 5× what the DB can comfortably take.

The fix is almost embarrassingly simple: add a little **randomness (jitter)** to every TTL so keys written together don't expire together.

LIB/TTL.JS COPY

```
// base TTL + up to `spread` (default 20%) random extra seconds
export const withJitter = (baseSeconds, spread = 0.2) =>
  baseSeconds + Math.floor(Math.random() * baseSeconds * spread);

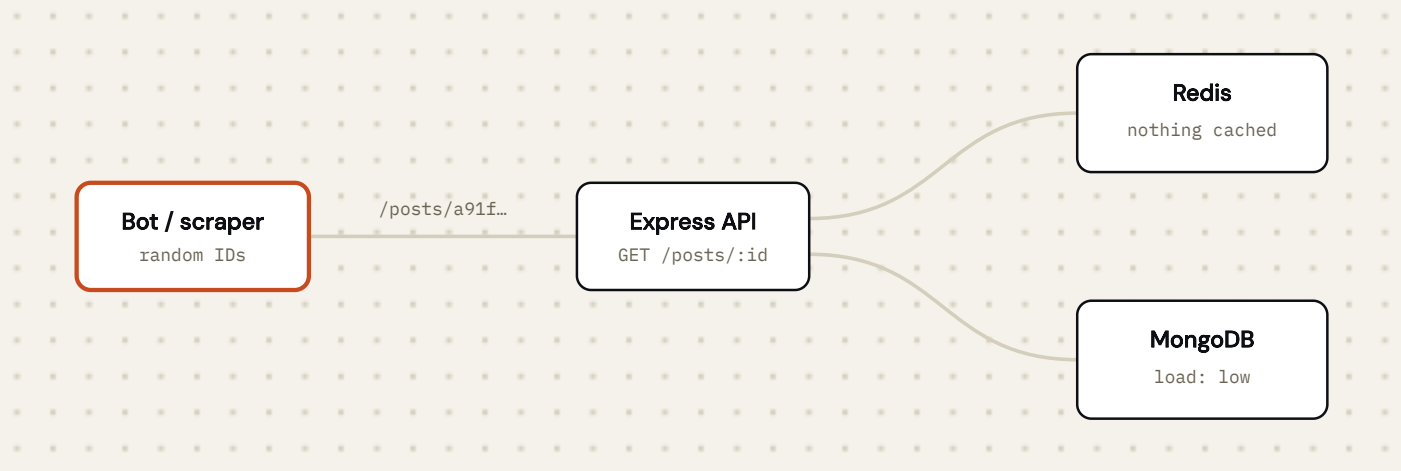
await redis.set(`product:${id}`, JSON.stringify(product), "EX", withJitter(3600));
// expires somewhere between 60 and 72 minutes, not all at 60:00
```

- **Cold cache after a Redis restart?** Turn on persistence (RDB/AOF) or use a managed service with replicas, so a restart doesn't wipe everything. Warm the most important keys before sending traffic.
- **Protect the DB anyway.** The stampede lock from Section 8 and rate limiting from Section 12 both limit how much load can reach MongoDB at once.

Penetration: asking for data that doesn't exist

Cache-aside only caches things it finds. If a bot (or a bug) keeps requesting `/posts/<random id>`, every request is a miss and every miss goes to the database, which finds nothing. Nothing gets cached, so the cache never helps. That's **cache penetration**.

STEP-THROUGH **Requests that slip straight through the cache**



A bot requests IDs that don't exist

A scraper loops over random IDs, or a buggy client keeps asking for a deleted post.

← BACK STEP 1 / 4 ▶ AUTO-PLAY NEXT →

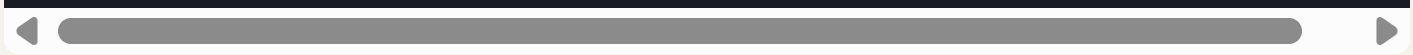
SERVICES/POST.SERVICE.JS · PENETRATION-SAFE COPY

```
import mongoose from "mongoose";
import { withJitter } from "../lib/ttl.js";

const NIL = "__nil__";

export async function getPost(postId) {
  if (!mongoose.isValidObjectId(postId)) return null;     // garbage never reaches the
  const key = `post:${postId}`;
  const cached = await redis.get(key);
  if (cached === NIL) return null;                        // known "doesn't exist"
  if (cached) return JSON.parse(cached);

  const post = await Post.findById(postId).lean();
  if (!post) {
    await redis.set(key, NIL, "EX", 30);                  // short TTL for negatives
    return null;
  }
  await redis.set(key, JSON.stringify(post), "EX", withJitter(60));
  return post;
}
```



💡 Keep the negative TTL short. If the post gets created a few seconds later, you don't want "not found" cached for an hour. For huge ID spaces, teams also use a **Bloom filter** (included in Redis 8, or Redis Stack on older versions). It answers "definitely doesn't exist" without storing every ID.

Problem	What happens	Fix
Stampede	One hot key expires and many requests rebuild it together	Lock (SET NX), single-flight, early refresh
Avalanche	Many keys expire at once, or the cache restarts empty	TTL jitter, persistence/replicas, warm-up
Penetration	Requests for missing data always miss	Cache nulls briefly, validate IDs, Bloom filter, rate limit

QUIZ · 09

10,000 product keys were all cached at 00:00 with `EX 3600`. What's the best quick fix to avoid an avalanche at 01:00?

A — Add random jitter to each key's TTL

B — Use a longer TTL of 7,200 for all of them

QUIZ · 09

A bot requests `/posts/<random id>` for IDs that never exist. Why doesn't a plain cache-aside setup help?

A — Nothing gets cached for missing data, so every request misses and goes to the DB

B — Redis can't store keys that contain random strings

10 Memory limits, eviction, and when Redis fails

RAM runs out, servers go down, and one key can get too hot

When memory is full: maxmemory and eviction

RAM is finite. You tell Redis how much it may use with `maxmemory`, and what to do when it's full with `maxmemory-policy`. For a pure cache, the usual choice is to let Redis evict the keys that matter least.

TERMINAL · RUN REDIS WITH A MEMORY CAP COPY

```
docker run -d --name redis -p 6379:6379 redis:7-alpine \
  redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
```

```
# or change it on a running server
docker exec -it redis redis-cli CONFIG SET maxmemory-policy allkeys-lru
```

INTERACTIVE **Eviction at maxmemory** click a key to GET it · capacity: 6 keys

SET A NEW KEY | START TRAFFIC | ALLKEYS-LRU | ALLKEYS-LFU | NOEVICTION

USED: 6 / 6 KEYS (MAXMEMORY)

post:42
idle 1s · hits 48

site:banner
idle 3s · hits 30

feed:9
idle 5s · hits 9

lb:weekly
idle 7s · hits 14

user:7
idle 11s · hits 4

otp:5521
idle 16s · hits 1

memory is full (6 / 6). Click a key to GET it, then SET a new key and see which one gets

POLICY	EVICTS	USE WHEN
`noeviction`	Nothing. Writes fail with an OOM error.	The default. Right when Redis holds data you can't lose (queues, sessions).
`allkeys-lru`	Least **recently** used key, from all keys	Pure cache. The best general default.
`allkeys-lfu`	Least **frequently** used key	A few keys are always hot (our viral post) and must survive bursts of one-off keys.
`volatile-lru` / `volatile-lfu`	Same, but only keys that **have a TTL**	One Redis holds both cache keys (with TTL) and permanent keys (without).
`volatile-ttl`	Key closest to expiring	Rarely the best choice.
`allkeys-random` / `volatile-random`	A random key	When every key matters equally.

💡 Redis LRU is **approximate**. It samples a handful of keys (`maxmemory-samples`, default 5) and evicts the best candidate among them. That's cheap and very close to true LRU. Real LFU also gives brand-new keys a small starting count and lets counts decay over time, so yesterday's hot key doesn't stay protected forever.

⚠️ If the same Redis also stores **rate-limit counters, sessions or leaderboards**, an `allkeys-*` policy may evict them to make room for cache entries. Either run a separate Redis for the cache, or use a `volatile-*` policy and give TTLs only to cache keys.

When Redis goes down

A cache should make the app faster, not become a new single point of failure. If Redis is unreachable, a well-written read **treats it as a miss** and goes to the database, and it fails fast instead of hanging.

LIB/CACHE.JS COPY

```
import { Redis } from "ioredis";

export const redis = new Redis(process.env.REDIS_URL, {
  enableOfflineQueue: false,   // don't queue commands while disconnected; reject immedi
  maxRetriesPerRequest: 1,
  commandTimeout: 200,         // a cache read slower than 200 ms counts as a miss
});
redis.on("error", (err) => console.warn("redis:", err.message));

export async function cacheGet(key) {
  try {
    return await redis.get(key);
  } catch {
    return null;               // Redis down or slow → behave like a miss
  }
}

export async function cacheSet(key, value, ttl) {
  try {
    await redis.set(key, value, "EX", ttl);
  } catch {
    /* losing a cache write is fine; the DB still has the data */
```

```
  }
}
```

⚠️ Falling back to the DB is right, but think it through: if Redis dies *during* viral traffic, all of that load lands on MongoDB, which is the Section 1 crash again. Production setups add replicas with automatic failover (Sentinel, Redis Cluster or a managed service), plus rate limiting and timeouts on the DB path.

Hot keys: one key, too much traffic

Even Redis has limits. A single viral key always lives on one Redis node, and lakhs of `GET post:42` calls per second all hit that one node's CPU and network. The fix is a tiny **in-process cache** (L1) with a 1–2 second TTL in front of Redis (L2). Each server then asks Redis for that key about once a second, instead of thousands of times.

LIB/HOTCACHE.JS COPY

```
const local = new Map(); // key -> { value, until }   (use the `lru-cache` package in pr

export async function getHot(key, ttlMs = 1000) {
  const hit = local.get(key);
  if (hit && hit.until > Date.now()) return hit.value;   // L1: process memory, ~0 ms

  const value = await cacheGet(key);                     // L2: Redis
  local.set(key, { value, until: Date.now() + ttlMs });
  return value;
}
```

QUIZ · 10

Redis hits `maxmemory` with the default `noeviction` policy. What happens on the next `SET`?

A — Redis returns an OOM error and refuses the write

B — Redis silently deletes the oldest key

QUIZ · 10

Redis is down. What should a well-written cache-aside read do?

A — Return a 500 error to the user

B — Treat it as a miss and read from the DB, using fail-fast timeouts

11 Redis data structures

Redis isn't just key → string. Pick the right shape for the job

Caching only needed strings. But a Redis value can be a richer structure, and each one comes with its own commands that run **inside Redis, atomically, with a known cost**. That's why the same Redis can also run rate limiters, presence tracking and leaderboards, which the rest of these notes cover.

Type	Think of it as	Use it for	Key commands	Cost
String	One value or a counter	Cached JSON, view counts, OTPs, locks, fixed-window limits	`SET` `GET` `INCR`	O(1)
Hash	A flat JS object	User profile, per-socket info, post stats	`HSET` `HGET` `HGETALL` `HINCRBY`	O(1) per field
List	An array you push and pop at both ends	Recent activity, latest N comments, simple queues	`LPUSH` `LRANGE` `LTRIM`	O(1) push/pop
Set	A bag of unique strings	Who liked a post, online users, tags	`SADD` `SISMEMBER` `SCARD`	O(1) add/check
Sorted set	Unique members, each with a score, always sorted	Leaderboards, sliding-window limits, trending	`ZADD` `ZINCRBY` `ZRANGE` `ZREVRANK`	O(log N)

Playground

This is a small Redis running in your browser, preloaded with sample keys. Pick a type, click a command chip (or type your own), and watch the keyspace on the right change. Every command also shows the matching **ioredis** call and what it returns in JavaScript.

STRINGS | HASHES | LISTS | SETS | SORTED SETS

Strings are the simplest type: one key, one value (text, a number, or serialised JSON, up to 512 MB). `INCR` treats the value as an integer and increments it atomically, so it's perfect for counters shared across servers.

Used in: **caching** · **view counters** · **locks** · **fixed-window rate limits**

`SET post:42 '{"title":"Viral post","likes":1200}' EX 60` `GET post:42`
`INCR post:42:views` `INCRBY post:42:views 10` `SET lock:post:42 req-7 NX PX 5000`
`TTL post:42` `GET user:42`

```
Sample keys loaded. Click a command chip, or type one below and press Enter. ↑ / ↓ recall history.

127.0.0.1:6379> type a command, e.g. INCR post:42:views                    RUN
```

KEYSPACE · 6 KEYS — RESET SAMPLE DATA

STRING post:42:views

1024

HASH user:42

name	Ankur
city	Bhopal
role	instructor

LIST feed:42

0 post:901 | 1 post:877 | 2 post:850

posts 12

SET `post:42:likers`

`user:7` `user:9` `user:15`

SET `online:users`

`user:9` `user:21`

ZSET `lb:weekly`

highest first · left column = ZREVRANK

0	kabir	150
1	aarav	120
2	diya	95
3	meera	80

💡 **ioredis return types to remember:** integer replies (`INCR`, `SADD`, `ZREVRANK`) come back as JS numbers. `HGETALL` comes back as a plain object. But scores from `ZSCORE` and `ZINCRBY` come back as **strings**, so wrap them in `Number()`. Try `GET user:42` to see the `WRONGTYPE` error you get when you use the wrong command for a key's type.

More types worth knowing

TYPE	ONE-LINE IDEA	TYPICAL USE
Pub/Sub	`PUBLISH` a message and every `SUBSCRIBE`d connection receives it instantly. Fire-and-forget: nothing is stored.	Socket.IO across servers (Section 14), cache-invalidation broadcasts
Streams	An append-only log with consumer groups and acknowledgements	Event pipelines and job queues that must not lose messages
HyperLogLog	Approximate unique counts using about 12 KB, however many items	Unique visitors per day
Bitmaps	One bit per user ID	Daily active flags, streaks
Geo	Members with latitude/longitude, searchable by radius	"Restaurants within 3 km"

QUIZ · 11

You need to check "has user:7 already liked post 42?" instantly, with no duplicate likes. Which structure fits?

A — Set: `SADD` to like, `SISMEMBER` to check

B — List: `LPUSH` to like, `LRANGE` to check

QUIZ · 11

Which structure keeps members ordered by a number and can tell you a member's rank in O(log N)?

A — Hash

B — Sorted set

12 Rate limiting Express APIs

One shared counter for every server, so the limit actually holds

The app survived the viral post. Now someone writes a script that hits `POST /auth/login` 500 times a second trying passwords, or scrapes `/posts/:id` for every ID. We need a rule like **"at most N requests per time window per IP (or per user)"**, and anything over the limit gets `429 Too Many Requests`.

The first version everyone writes

MIDDLEWARE/NAIVELIMIT.JS · DON'T SHIP THIS COPY

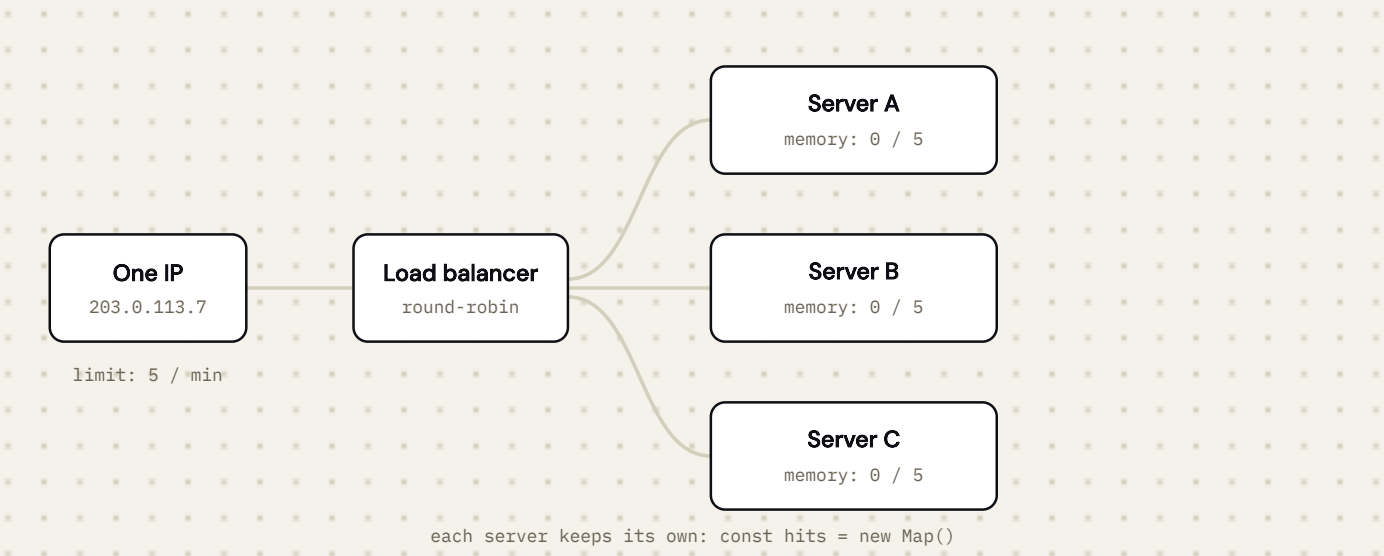
```
const hits = new Map(); // ip -> { count, resetAt }

export function naiveLimit(req, res, next) {
  const now = Date.now();
  const entry = hits.get(req.ip) ?? { count: 0, resetAt: now + 60_000 };
  if (now > entry.resetAt) { entry.count = 0; entry.resetAt = now + 60_000; }
  entry.count++;
  hits.set(req.ip, entry);
```

```
  if (entry.count > 5) return res.status(429).send("Too many requests");
  next();
}
```

It works on your laptop, then production runs three copies of your server behind a load balancer.

STEP-THROUGH **Why an in-memory counter breaks with more than one server**



The rule: 5 requests per minute per IP

Each server runs the naive middleware, so each one counts in its own memory.

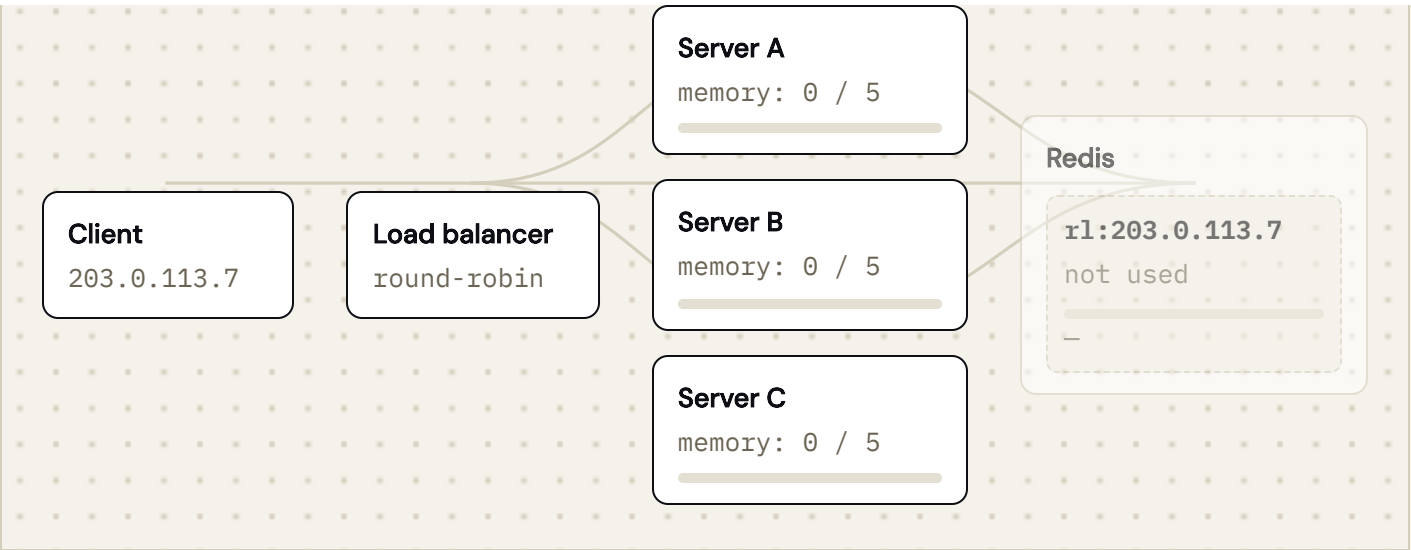
← BACK STEP 1 / 5 ▶ AUTO-PLAY NEXT →

Try it: in-memory vs Redis counter

Limit: **5 requests per 10 seconds**. Hit *Spam ×12* in each mode and compare how many get through.

SIMULATOR **One IP, three servers** limit: 5 requests / 10 s

SEND REQUEST SPAM ×12 IN EACH SERVER SHARED IN REDIS RESET



Allowed **0** · Blocked **0** · in the current window: **0** allowed

Fixed window with `INCR` + `EXPIRE`

- **`INCR rl:login:<ip>`**
  Creates the key at 1 if it doesn't exist and returns the new count.
- **`EXPIRE key 60 NX`**
  Starts the 60 s window, but only if the key has no TTL yet (first request). Needs Redis 7+.
- **count > limit → 429, else next()**
  When the key expires, the next request starts a fresh window at 1.

MIDDLEWARE/RATELIMIT.JS COPY

```
import { redis } from "../lib/redis.js";

export function rateLimit({ limit = 100, windowSec = 60, prefix = "rl" } = {}) {
  return async (req, res, next) => {
    const key = `${prefix}:${req.ip}`;
    try {
      const [[, count], , [, ttl]] = await redis
        .multi()
        .incr(key)                      // +1 for this request
        .expire(key, windowSec, "NX")   // start the window only on the first hit
        .ttl(key)
        .exec();                        // → [[null, 3], [null, 0], [null, 42]]
```

```
      res.set("RateLimit-Limit", String(limit));
      res.set("RateLimit-Remaining", String(Math.max(0, limit - count)));

      if (count > limit) {
        res.set("Retry-After", String(ttl));
        return res.status(429).json({ message: "Too many requests. Slow down." });
      }
      next();
    } catch (err) {
      next(); // Redis down: fail open here. For /login you may prefer to fail closed.
    }
  };
}

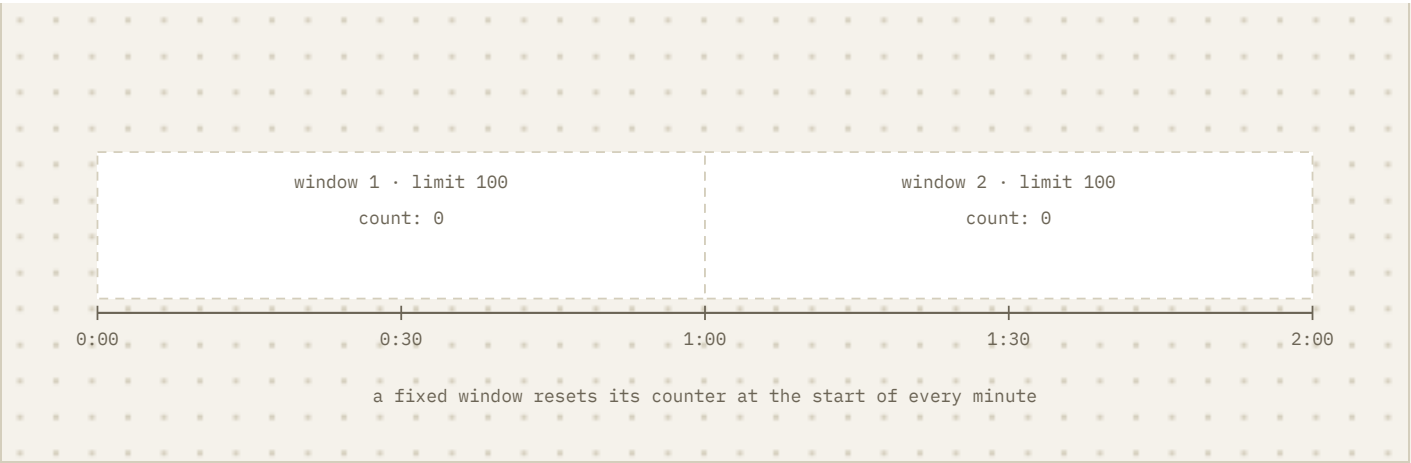
// usage
app.post("/auth/login", rateLimit({ limit: 5, windowSec: 60, prefix: "rl:login" }), logi
app.use("/api", rateLimit({ limit: 300, windowSec: 60, prefix: "rl:api" }));
```

⚠️ **Why `MULTI`?** If you send `INCR` and `EXPIRE` as two separate calls and the server crashes (or the connection drops) between them, the key exists with **no TTL**. That counter never resets and the user is blocked forever. `MULTI … EXEC` sends them as one transaction that runs together. On Redis older than 7 (no `NX` flag on `EXPIRE`), use a small Lua script instead.

⚠️ **Behind Nginx, an AWS ALB or a Kubernetes ingress, `req.ip` is the proxy's IP** unless you set `app.set("trust proxy", 1)`. Without it, every user shares a single counter and you'll block everyone at once.

The weakness of fixed windows, and the sliding window

STEP-THROUGH **The boundary burst**



Fixed window

The limit is 100 per minute. The counter belongs to a clock window (0:00–1:00, 1:00–2:00) and resets when a new one starts.

← BACK — STEP **1** / 4 — ▶ AUTO-PLAY — NEXT →

A **sliding window log** stores each request's timestamp as a sorted-set member, with the timestamp as its score. Each request removes entries older than the window, adds itself, and counts what's left.

LIB/SLIDINGWINDOW.JS COPY

```
import { randomUUID } from "node:crypto";
import { redis } from "./redis.js";

export async function slidingWindow(key, limit, windowMs) {
  const now = Date.now();
  const results = await redis
    .multi()
    .zremrangebyscore(key, 0, now - windowMs)   // 1. drop requests older than the windo
    .zadd(key, now, `${now}:${randomUUID()}`)    // 2. record this request (unique membe
    .zcard(key)                                  // 3. how many are inside the window?
    .pexpire(key, windowMs)                      // 4. idle keys clean themselves up
    .exec();

  const count = results[2][1];
  return { allowed: count <= limit, count };
}

// usage: strict limit for OTP requests
```

```
const { allowed } = await slidingWindow(`rl:otp:${phone}`, 3, 10 * 60_000);
if (!allowed) return res.status(429).json({ message: "Too many OTP requests" });
```

💡 This version also records rejected attempts, so a client that keeps hammering stays blocked. That's usually what you want for login and OTP. It stores one entry per request, so use it for small, strict limits and keep the fixed window for high-volume API limits.

Production shortcut: express-rate-limit + rate-limit-redis

For everyday API limits, use the well-tested `express-rate-limit` middleware and point its store at Redis. Every server then shares the same counters.

APP.JS COPY

```
// npm i express-rate-limit rate-limit-redis
import { rateLimit } from "express-rate-limit";
import { RedisStore } from "rate-limit-redis";
import { redis } from "./lib/redis.js";

app.set("trust proxy", 1); // real client IP behind a load balancer

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,      // 15 minutes
  limit: 300,                    // per IP per window (older versions call this `max`)
  standardHeaders: "draft-7",    // send RateLimit-* headers
  legacyHeaders: false,
  store: new RedisStore({
    prefix: "rl:api:",
    sendCommand: (command, ...args) => redis.call(command, ...args),
  }),
});

const loginLimiter = rateLimit({
  windowMs: 60 * 1000,
  limit: 5,
  message: { message: "Too many login attempts. Try again in a minute." },
  store: new RedisStore({
    prefix: "rl:login:",         // each limiter needs its own prefix
    sendCommand: (command, ...args) => redis.call(command, ...args),
  }),
});
```

```
app.use("/api", apiLimiter);
app.post("/auth/login", loginLimiter, loginController);
```

OPTION	VALUE USED	WHAT IT DOES
windowMs	`15 * 60 * 1000`	Length of the window in milliseconds.
limit	`300`	Requests allowed per key (IP by default) per window.
standardHeaders	`"draft-7"`	Tells clients their limit and remaining requests through standard headers.
store	`new RedisStore(…)`	Moves the counters from process memory to Redis. This is what makes the limit hold across servers.
sendCommand	`redis.call(…)`	How the store talks to Redis. `call()` is ioredis's "run any raw command" method.
prefix	`"rl:login:"`	Keeps the login limiter's keys separate from the general API limiter's.

QUIZ · 12

3 Express servers behind a round-robin load balancer each count requests in memory with a limit of 10/min. How many requests can one IP really make per minute?

A — 10

B — Up to 30

QUIZ · 12

Why wrap `INCR` and `EXPIRE` in `MULTI`?

A — So a crash between the two can't leave a counter with no expiry, blocking the user forever

B — Because INCR doesn't work outside a transaction

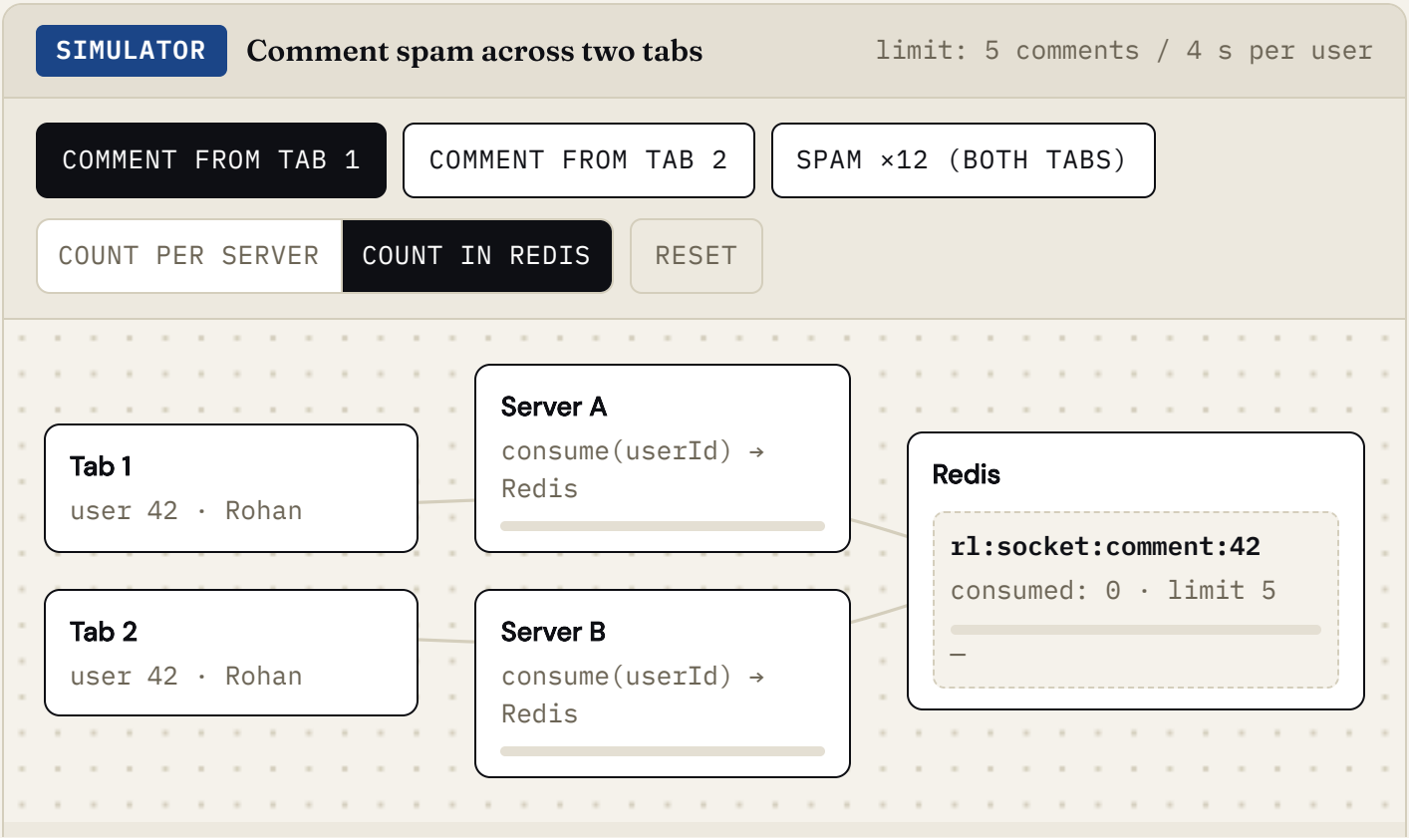
13 Rate limiting Socket.IO events

Express middleware never sees socket events, so limit them per user in Redis

The viral post has live comments over Socket.IO. A WebSocket is one long-lived connection. The HTTP rate limiter runs once, at the handshake, and after that every `socket.emit("comment:new", …)` flows through the open socket without touching Express. One script can open a socket and emit 1,000 comments a second, flooding every viewer in the room.

- **Limit events, not just connections.** Use `socket.use()`, which runs for every incoming packet on that socket.
- **Key by user, store in Redis.** The same user can open three tabs, and the load balancer may send each tab to a different server. Per-process counters would let them send three times the limit.
- **Also limit handshakes** per IP with `io.use()`, so nobody can open thousands of sockets.

Try it: two tabs, two servers, one user



Meera this is the best post on my feed today 😂

Kabir Bhopal walo, attendance lagao

With rate-limiter-flexible (recommended)

SOCKET/LIMITS.JS COPY

```
// npm i rate-limiter-flexible
import { RateLimiterRedis } from "rate-limiter-flexible";
import { redis } from "../lib/redis.js";

const commentLimiter = new RateLimiterRedis({
  storeClient: redis,
  keyPrefix: "rl:socket:comment",
  points: 5,        // 5 comments…
  duration: 4,      // …per 4 seconds, per user
});

io.on("connection", (socket) => {
  const userId = socket.data.userId;            // set by your auth middleware in io.use

  socket.use(async ([event], next) => {
    if (event !== "comment:new") return next();  // only limit what needs it
    try {
      await commentLimiter.consume(userId);       // rejects when out of points
      next();
    } catch (rej) {
      if (rej instanceof Error) return next();    // Redis error → let it through
      socket.emit("rate_limited", { event, retryAfterMs: rej.msBeforeNext });
      // not calling next() drops this packet
    }
  });

  socket.on("comment:new", (comment) => {
    io.to(`post:${comment.postId}`).emit("comment:added", comment);
```

```
  });
});
```

The same idea with plain ioredis

SOCKET/LIMITS.JS · NO LIBRARY COPY

```
async function allow(key, limit, windowSec) {
  const [[, count]] = await redis.multi().incr(key).expire(key, windowSec, "NX").exec();
  return count <= limit;
}

socket.use(async ([event], next) => {
  if (await allow(`rl:socket:${socket.data.userId}:${event}`, 5, 4)) return next();
  socket.emit("rate_limited", { event });
});
```

Limit connection attempts too

SOCKET/LIMITS.JS COPY

```
const connectLimiter = new RateLimiterRedis({
  storeClient: redis, keyPrefix: "rl:socket:connect", points: 10, duration: 60,
});

io.use(async (socket, next) => {
  const ip = socket.handshake.headers["x-forwarded-for"]?.split(",")[0] ?? socket.handsh
  try {
    await connectLimiter.consume(ip);
    next();
  } catch (rej) {
    if (rej instanceof Error) return next();
    next(new Error("Too many connection attempts"));   // client receives "connect_erro
  }
});
```

CLIENT.JS COPY

```
socket.on("rate_limited", ({ retryAfterMs }) => {
  toast(`Slow down! Try again in ${Math.ceil((retryAfterMs ?? 1000) / 1000)}s`);
});
socket.on("connect_error", (err) => console.warn(err.message));
```

QUIZ · 13

Why doesn't `express-rate-limit` protect against a client spamming `socket.emit("comment:new")`?

A — Socket events travel over the already-open WebSocket and never pass through Express middleware

B — express-rate-limit only works with GET requests

QUIZ · 13

A user opens 3 tabs, and each tab connects to a different Socket.IO server. Where should the comment counter live so the limit holds?

A — In each server's memory, keyed by socket.id

B — In Redis, keyed by user ID

14 Socket.IO across many servers

Redis Pub/Sub carries broadcasts between servers, and sets and hashes store presence

One Socket.IO server can't hold every live viewer of a viral post, so you run two or more. Here's the catch: **each server only knows the sockets connected to it.** `io.to("post:42").emit(…)` on server 1 reaches

server 1's sockets and nobody else.

STEP-THROUGH **A live comment that only half the room sees**



Two servers, one room

The load balancer put Alice and Bob on server 1, and Charlie and Dia on server 2. All four joined the room `post:42` for live comments.

← BACK STEP **1** / 5 ▶ AUTO-PLAY NEXT →

KEY CONCEPT

Pub/Sub: `PUBLISH channel message` delivers the message instantly to every connection that ran `SUBSCRIBE channel`. It's fire-and-forget. Nothing is stored, so a server that's offline at that moment misses the message.

A connection in subscribe mode can only run Pub/Sub commands. That's why the adapter needs a **second client**, made with `pubClient.duplicate()`.

Pub/Sub in ioredis, by itself

PUBSUB-DEMO.JS COPY

```
const sub = redis.duplicate();                  // dedicated subscriber connection
await sub.subscribe("post:42:events");
```

```
sub.on("message", (channel, message) => {
  console.log(channel, JSON.parse(message));   // post:42:events { type: 'like', by: 'us
});

await redis.publish("post:42:events", JSON.stringify({ type: "like", by: "user:7" }));
// returns the number of subscribers that received it
```

The Socket.IO Redis adapter

SERVER.JS COPY

```
// npm i socket.io @socket.io/redis-adapter ioredis
import { createServer } from "node:http";
import { Server } from "socket.io";
import { createAdapter } from "@socket.io/redis-adapter";
import { Redis } from "ioredis";
import app from "./app.js";

const pubClient = new Redis(process.env.REDIS_URL);
const subClient = pubClient.duplicate();       // subscriber mode needs its own connecti

const httpServer = createServer(app);
const io = new Server(httpServer, {
  adapter: createAdapter(pubClient, subClient),
});

io.on("connection", (socket) => {
  socket.on("post:join", (postId) => socket.join(`post:${postId}`));
  socket.on("comment:new", (c) => {
    io.to(`post:${c.postId}`).emit("comment:added", c); // now reaches EVERY server
  });
});

httpServer.listen(process.env.PORT ?? 3000);
```

⚠️ **You still need sticky sessions.** Socket.IO can start with HTTP long-polling before upgrading to WebSocket, and those polling requests must reach the same server, or you'll see HTTP 400 "Session ID unknown". Enable stickiness on the load balancer (ALB target-group stickiness, nginx `ip_hash`), or connect with `io(url, { transports: ["websocket"] })`.

💡 On Redis 7+, the Socket.IO docs recommend the **sharded adapter** (`createShardedAdapter`, built on sharded Pub/Sub) for new projects, especially with Redis Cluster. With an adapter in place, `io.fetchSockets()` and `io.in(room).fetchSockets()` work across servers too. A background worker without a Socket.IO server can emit events through Redis with `@socket.io/redis-emitter`.

Central storage: who's online right now?

The adapter only *relays* messages. It doesn't give your REST API an answer to "how many people are watching?" or "is user 9 online?". Keep that state in Redis yourself with a **hash** (sockets per user, since one user can have several tabs) and a **set** (online users).

SOCKET/PRESENCE.JS COPY

```
io.on("connection", async (socket) => {
  const userId = socket.data.userId;

  // count this user's open sockets (tabs, phone + laptop…)
  const tabs = await redis.hincrby("presence:sockets", userId, 1);
  if (tabs === 1) await redis.sadd("online:users", userId);   // first socket → online

  socket.on("disconnect", async () => {
    const left = await redis.hincrby("presence:sockets", userId, -1);
    if (left <= 0) {
      await redis.multi()
        .hdel("presence:sockets", userId)
        .srem("online:users", userId)
        .exec();
    }
  });
});

// anywhere: a REST route, another server, a cron job
const onlineCount = await redis.scard("online:users");              // 1284
const isOnline = await redis.sismember("online:users", userId);     // 1 or 0
```

⚠️ If a server crashes, its `disconnect` handlers never run, and those users look online forever ("ghost users"). Production setups add a heartbeat: each server refreshes a key like `presence:server:<id>` with a short TTL and cleans up users belonging to servers whose key expired. Or each user gets `presence:user:<id>` with `EX 60`, refreshed by a client ping.

QUIZ · 14

Why does the Redis adapter need two ioredis clients (pub and sub)?

A — A connection in SUBSCRIBE mode can only run Pub/Sub commands, so publishing needs a separate connection

B — One client is a backup in case the other fails

QUIZ · 14

With the Redis adapter set up, does Redis store the list of rooms and sockets?

A — Yes, Redis keeps all room membership permanently

B — No. It only relays messages through Pub/Sub, so you store presence yourself with sets and hashes

15 Real-time leaderboards with sorted sets

Millions of scores, always ranked, updated in O(log N)

Kodr runs a weekly coding contest: 50,000 participants, scores changing every few seconds, and everyone refreshing to see the **top 10** and **"my rank"**. In SQL, "my rank" is `SELECT COUNT(*) + 1 FROM scores WHERE score > :myScore`. That runs on every refresh, for every participant, while scores keep changing underneath it. It's fine for 500 users and painful for 50,000.

KEY CONCEPT

A **sorted set** keeps unique members ordered by score *at all times*. Internally it pairs a skip list (for ordering) with a hash table (for member → score lookups). Redis never "sorts" when you ask for the leaderboard, because it's already sorted.

`ZINCRBY` · `ZREVRANK` · `ZSCORE` cost O(log N) or better. Top-10 with `ZRANGE … REV` costs O(log N + 10). With 50,000 members, $\log_2 N$ is about 16 steps.

LIVE DEMO **Kodr weekly contest** · `lb:contest:42` rows move by ZREVRANK

START LIVE CONTEST | ROHAN SOLVES A PROBLEM · +60 | RESET

#1	K	Kabir	510
#2	V	Vikram	460
#3	A	Aarav	420
#4	D	Diya	390
#5	R	Rohan (you)	350
#6	M	Meera	300
#7	S	Sana	280
#8	I	Isha	240

YOUR RANK

`ZREVRANK lb:contest:42 rohan`

#5 (integer) 4 → 0-based, so +1

GET /LEADERBOARD/TOP?LIMIT=3

```
[
  {"rank":1,"userId":"kabir","score":510},
  {"rank":2,"userId":"vikram","score":460},
```

```
  {"rank":3,"userId":"aarav","score":420}
]
```

```
ZADD lb:contest:42 420 aarav 390 diya … → (integer) 8
```

The commands a leaderboard needs

COMMAND	LEADERBOARD JOB	RETURNS (IOREDIS)	COST
`ZADD lb 120 aarav`	Set a score (create or overwrite)	1 if new member, 0 if updated	O(log N)
`ZINCRBY lb 60 rohan`	Add points atomically	new score as a **string**: "410"	O(log N)
`ZRANGE lb 0 9 REV WITHSCORES`	Top 10, highest first	flat array: [m1, s1, m2, s2, …]	O(log N + 10)
`ZREVRANK lb rohan`	My position (0 = first)	number, or null if absent	O(log N)
`ZSCORE lb rohan`	My score	string, or null	O(1)
`ZCARD lb`	Number of participants	number	O(1)
`ZREM lb cheater`	Remove someone	1 or 0	O(log N)

💡 `ZRANGE … REV` needs Redis 6.2+. On older servers, use `ZREVRANGE lb 0 9 WITHSCORES`, which still works but is marked deprecated. Members with **equal scores** are ordered by member name (lexicographically, and reversed with `REV`). If "whoever got there first wins" matters, fold a time component into the score.

The API

ROUTES/LEADERBOARD.ROUTES.JS COPY

```
import { Router } from "express";
import { redis } from "../lib/redis.js";
```

```
const router = Router();
const LB = "lb:contest:42";

// flat [member, score, member, score…] → [{ rank, userId, score }]
const toRows = (flat, startRank = 1) => {
  const rows = [];
  for (let i = 0; i < flat.length; i += 2) {
    rows.push({ rank: startRank + i / 2, userId: flat[i], score: Number(flat[i + 1]) });
  }
  return rows;
};

// a user solved a problem
router.post("/score", async (req, res) => {
  const { userId, points } = req.body;
  const total = await redis.zincrby(LB, points, userId);      // "410" (a string)
  res.json({ userId, total: Number(total) });
});

// top N, highest first
router.get("/top", async (req, res) => {
  const limit = Math.min(Number(req.query.limit) || 10, 100);
  const flat = await redis.zrange(LB, 0, limit - 1, "REV", "WITHSCORES");
  res.json(toRows(flat));
});

// my rank + the 2 people above and below me
router.get("/me/:userId", async (req, res) => {
  const { userId } = req.params;
  const rank = await redis.zrevrank(LB, userId);              // 0-based, or null
  if (rank === null) return res.json({ rank: null });

  const start = Math.max(0, rank - 2);
  const [flat, score] = await Promise.all([
    redis.zrange(LB, start, rank + 2, "REV", "WITHSCORES"),
    redis.zscore(LB, userId),
  ]);
  res.json({ rank: rank + 1, score: Number(score), around: toRows(flat, start + 1) });
});

export default router;
```

Names for the top 10 without 10 DB queries

The sorted set only stores user IDs. Keep display names in hashes and fetch them all in one round trip with a **pipeline**, which batches commands without making them a transaction.

ROUTES/LEADERBOARD.ROUTES.JS COPY

```
const top = toRows(await redis.zrange(LB, 0, 9, "REV", "WITHSCORES"));

const pipe = redis.pipeline();
top.forEach((row) => pipe.hget(`user:${row.userId}`, "name"));
const replies = await pipe.exec();                // [[null, "Kabir"], [null, "Aarav"], …

top.forEach((row, i) => (row.name = replies[i][1] ?? "Unknown"));
```

Weekly and monthly boards that clean themselves up

SERVICES/SCORE.SERVICE.JS COPY

```
export async function addPoints(userId, points, week /* e.g. "2026-W43" */) {
  const key = `lb:weekly:${week}`;
  await redis
    .multi()
    .zincrby(key, points, userId)
    .zincrby("lb:all-time", points, userId)
    .expire(key, 60 * 60 * 24 * 14, "NX")    // weekly board deletes itself after 2 weeks
    .exec();
}

// combine weeks into a monthly board (sums scores by default)
await redis.zunionstore("lb:month:2026-10", 4,
  "lb:weekly:2026-W40", "lb:weekly:2026-W41", "lb:weekly:2026-W42", "lb:weekly:2026-W43"
```

💡 To make it truly live, emit a `leaderboard:update` event over Socket.IO after each `ZINCRBY`, throttled to about once a second. With the Redis adapter from Section 14, it reaches viewers on every server.

QUIZ · 15

`ZREVRANK lb kabir` returns `0`. What is Kabir's position?

A — Last place

B — 1st place. Ranks are 0-based and REV puts the highest score first

QUIZ · 15

Why is a sorted set better than `SELECT COUNT(*)` for "my rank" at scale?

A — Redis keeps members sorted as scores change, so a rank lookup is O(log N) instead of counting rows every time

B — Sorted sets store data on disk, so they're more durable

16 Cheat sheet & production checklist

Which structure for which job, and what to check before you ship

Use case	Structure	Key pattern	Commands
Cache an object	String	`post:{id}`	`SET … EX` · `GET` · `DEL`
Stampede lock	String	`lock:{key}`	`SET … NX PX` · Lua compare-and-DEL
Counters (views, likes)	String	`post:{id}:views`	`INCR` · `INCRBY`

Use case	Structure	Key pattern	Commands
Rate limit, fixed window	String	`rl:{scope}:{ip}`	`MULTI` · `INCR` · `EXPIRE NX`
Rate limit, sliding window	Sorted set	`rl:{scope}:{id}`	`ZREMRANGEBYSCORE` · `ZADD` · `ZCARD`
Object with fields	Hash	`user:{id}`	`HSET` · `HGET` · `HGETALL` · `HINCRBY`
Recent activity feed	List	`feed:{userId}`	`LPUSH` · `LTRIM` · `LRANGE`
Likers / online users	Set	`post:{id}:likers`	`SADD` · `SISMEMBER` · `SCARD`
Leaderboard	Sorted set	`lb:{period}`	`ZINCRBY` · `ZRANGE REV` · `ZREVRANK`
Messages between servers	Pub/Sub	channel names	`PUBLISH` · `SUBSCRIBE`

Housekeeping commands

REDIS-CLI COPY

```
127.0.0.1:6379> INFO stats            # keyspace_hits / keyspace_misses → hit ratio
127.0.0.1:6379> INFO memory           # used_memory_human, maxmemory
127.0.0.1:6379> SLOWLOG GET 10        # the slowest recent commands
127.0.0.1:6379> TYPE user:42          # which structure is this key?
127.0.0.1:6379> SCAN 0 MATCH post:* COUNT 100   # iterate keys safely
127.0.0.1:6379> UNLINK post:42        # delete without blocking (frees memory in the bac
```

SCRIPTS/CLEARPOSTCACHE.JS COPY

```
// Never run KEYS * on production: it blocks Redis while it walks every key.
const stream = redis.scanStream({ match: "post:*", count: 100 });
stream.on("data", async (keys) => {
```

```
  if (keys.length) await redis.unlink(...keys);
});
stream.on("end", () => console.log("post cache cleared"));
```

Production checklist

- **One client per process**
  Import it everywhere. Create a `duplicate()` only for subscribers.
- **Every cache key has a TTL, with jitter**
  No "forever" cache entries. A plain `SET` wipes the TTL.
- **Set `maxmemory` and a policy**
  Keep pure cache separate from counters, sessions and leaderboards.
- **Invalidate on write: update the DB, then `DEL`**
  Know which keys each write affects.
- **Protect hot keys**
  Lock with `SET NX PX`, single-flight, optional L1 cache.
- **Fail fast, fall back**
  `commandTimeout`, `enableOfflineQueue: false`, try/catch around cache calls.
- **Atomic multi-step logic**
  `MULTI` / `EXEC` or Lua for anything like "INCR then EXPIRE".
- **No `KEYS *`, no huge values**
  Use `SCAN`. Big values (hundreds of KB) block the single thread for everyone.
- **Real client IPs**
  `app.set("trust proxy", 1)` before any IP-based limiter.
- **Lock it down**
  Password/ACL, TLS on managed services, never exposed to the internet (security groups).
- **Watch it**
  Hit ratio, memory, evictions (`evicted_keys`) and the slow log.

QUIZ · 16

You need to delete all `post:*` keys on a production Redis. Which approach is right?

A — `KEYS post:*`, then DEL them all

B — SCAN in batches (ioredis `scanStream`) and UNLINK each batch

QUIZ · 16

Which `INFO` fields give you the cache hit ratio?

A — `keyspace_hits` and `keyspace_misses`

B — `used_memory` and `maxmemory`